

Number System

how computers represent everything using only 0s and 1s

What Is a Number System?

DEFINITION

A number system is a way of representing numbers using a **fixed set of digits** and a **base**.

The base tells you how many digits the system has. Decimal has ten digits, so after 9 you roll over to 10. Binary has only two, so after 1 you roll over to 10 — which means **two**, not ten.

Binary

Base 2 — digits 0, 1

Octal

Base 8 — digits 0 to 7

Decimal

Base 10 — digits 0 to 9 (what humans use)

Hexadecimal

Base 16 — digits 0 to 9 and A to F
(A=10 ... F=15)

The Binary Number System

Binary is base 2. Only two digits exist: **0** and **1**. Computers use binary because their circuits only recognise two electrical states — **off** and **on**.

In simple words: a computer is built from millions of tiny electronic switches called **transistors**. Groups of switches form **logic gates**, and logic gates combine to build the processor. Each switch is only ever off or on — which is exactly what 0 and 1 represent.



Binary is not a design choice someone made to be difficult. It is forced by physics — a switch is either carrying current or it is not. Everything else in computing is built on top of that one fact.

Binary to Decimal

Give each digit a place value — powers of 2, starting from 2^0 on the right. Then add up only the places where the digit is 1.

1 $2^3 = 8$	0 $2^2 = 0$	1 $2^1 = 2$	1 $2^0 = 1$
-----------------------	-----------------------	-----------------------	-----------------------

$$8 + 0 + 2 + 1 = 11$$

$$1011_2 = 11_{10}$$

Decimal to Binary

Divide by 2 repeatedly, note the remainder each time, and then read the remainders **from bottom to top**.

$13 \div 2 = 6$ remainder **1**

$6 \div 2 = 3$ remainder **0**

$3 \div 2 = 1$ remainder **1**

$1 \div 2 = 0$ remainder **1**

read remainders bottom $\rightarrow$ top

$13_{10} = 1101_2$

⚠ Common mistake: reading the remainders top to bottom. That gives 1011, which is 11 — the wrong answer. Always read upward.

Bits, Bytes and Storage Units

A **bit** is one binary digit, a single 0 or 1. Group **8 bits** together and you get **1 byte**. Everything scales up from there.

8 Bits

$\rightarrow$

1 Byte

$\rightarrow$

KB

$\rightarrow$

MB

$\rightarrow$

GB

$\rightarrow$

TB

UNIT

EQUAL TO

1Byte

8 bits

1KB

1,024 Bytes

1MB

1,024 KB

1GB

1,024 MB

UNIT	EQUAL TO
1TB	1,024 GB

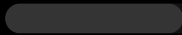


Why 1,024 and not 1,000? Because 1,024 is 2^{10} , a clean power of two. Storage manufacturers sometimes use decimal units instead (1 KB = 1,000 Bytes), which is why a "1TB" drive always shows up smaller than expected on your computer.

32-bit vs 64-bit

This describes the size of data the CPU can work with natively — which in turn sets how much memory the system can address.

32-bit



Much smaller address space — roughly 4 GB of memory can be addressed.

64-bit



Vastly larger address space — able to address far more memory. This is why modern computers mostly use 64-bit architectures.

This is the reason a 32-bit system cannot use 16 GB of RAM even if you physically install it. The CPU simply has no way to address that much.

SUMMARY

- Binary (2), Octal (8), Decimal (10), Hexadecimal (16).
- Binary to decimal: add the place values where the digit is 1.
- Decimal to binary: divide by 2, read remainders bottom to top.
- 8 bits = 1 byte; each step up is $\times 1,024$.
- 32-bit vs 64-bit is about how much memory the CPU can address.